Final_Report

May 16, 2026

30-Day Readmission Risk Prediction

Alejandro Barajas, Sri Ramani Thungapati, Himanshu Singh Rao

Introduction

In this report, we focused on the risk of high unplanned readmissions to a hospital within 30 days. It's essential to consider readmission because it can reflect the failure of initial treatments, help hospitals improve healthcare quality, reduce costs, and efficiently prepare the expected number of beds for people who are more than likely to be readmitted. To do this, we must implement a classification model, as the primary goal is to identify patients at high risk of readmission.

High-level framework of solution

To address this problem, our team followed four major stages of development. Stage 1 focused on building a strong foundation: each team member researched hospital readmission literature, conducted exploratory data analysis, and prepared the dataset for modeling. Stage 2 involved splitting the data into training, validation, and test sets, followed by implementing four baseline models—logistic regression, random forest, gradient boosting, and XGBoost. Based on their baseline performance, we selected logistic regression and XGBoost as the two models to tune and advance to Stage 3. Stage 3 consisted of optimizing these two models and comparing their performance to determine the stronger final model. Stage 4, the final stage, evaluated the models using classification metrics such as F1-score, F2-score, Brier score, ROC-AUC, confusion matrix, and PR-AUC. We then generated key visualizations and interpreted the results to draw meaningful conclusions.

Implementation

For this project, we use a hospital dataset from Kaggle. Our dataset contains 17 variables and 8000 rows, where each row represents a record of a hospital patient. The 17 variables are presented as follows:

patient_id - Numerical unique patient identifier

admission_date - Hospital admission date

season - Season of when the patient was admitted to the hospital

age - Age of the patient

gender - The gender of the patient

region - Region of the hospital that the patient was admitted to

primary_diagnosis - The main diagnosis of the patient

comorbidities_count - The number of additional conditions

length_of_stay - The number of days spent in the hospital

treatment_type - The type of treatment received for diagnosis

medications_count - The number of medications prescribed

followup_visits_last_year - Outpatient visits in the past year

prev_readmissions - Previous hospital admissions

insurance_type - The type of insurance the patient has

discharge_disposition - The type of discharge disposition the patient received after treatment

readmission_risk_score - The score a person has for readmission

Label - 1 if the patient has been readmitted within 30-days and 0 if the patient hasn't been readmitted within 30-days

We first conducted an exploratory data analysis to identify any initial issues and confirmed that all columns had appropriate data types, with no missing or empty values. However, we identified three variables that were not relevant to our study. So readmission_risk_score, admission_date, and patient_id were removed from the dataset.

We then prepared the data by applying binary encoding to the gender variable and constructing separate preprocessing pipelines for numerical and categorical features. These were combined into a unified preprocessing pipeline in which numerical variables were standardized and categorical variables were one-hot encoded. Finally, we performed a stratified 70% training, 15% validation, 15% test split to ensure that the class imbalance in the target label was preserved across all subsets.

For this classification task, we selected four appropriate baseline models: Logistic Regression, Random Forest, Gradient Boosting, and XGBoost. Logistic Regression provides a strong and interpretable baseline, performing reliably when relationships are linear or approximately linear, making it suitable for binary classification. Random Forest offers a nonlinear alternative, capturing feature interactions without extensive tuning and resisting overfitting on a dataset of roughly 8,000 observations. Gradient Boosting typically delivers strong predictive performance by sequentially correcting errors and capturing subtle patterns in the data, which is effective at this dataset size. Finally, XGBoost provides substantial performance potential due to its advanced regularization, efficient implementation, and numerous hyperparameter spaces, making it a powerful option for classification tasks. Together, these four models form a balanced set of baselines that combine interpretability, robustness, and performance potential, allowing for meaningful comparison and subsequent optimization.

The four baseline models were implemented using the same preprocessing pipeline and stratified 70/15/15 train, validation, and test split to ensure fair comparison across all approaches. Each model was trained on the same transformed dataset in which numerical variables were standardized using StandardScaler and categorical variables were one-hot encoded using OneHotEncoder. Model performance was evaluated on the validation and test sets using ROC-AUC, PR-AUC, Brier score, confusion matrix results, and classification metrics.

The baseline logistic regression model served as the primary linear classification baseline. Logistic regression was selected because it is highly interpretable and performs well when relationships between predictors and the target are mostly linear or monotonic. This was appropriate for our dataset

because variables such as age, comorbidities count, medications count, and previous readmissions generally increase alongside readmission risk. To better handle class imbalance, the baseline logistic regression model used class_weight="balanced", allowing the model to place greater emphasis on correctly classifying the minority class.

[ ]:
```
lr_pipe = Pipeline([
    ("preprocess", make_preprocessor()),
    ("clf", LogisticRegression(
        max_iter=2000,
        class_weight="balanced",
        random_state=RANDOM_STATE
    )),
])
```

The random forest baseline model provided a nonlinear alternative capable of capturing feature interactions and complex relationships without requiring extensive preprocessing assumptions. Random forest combines multiple decision trees through bagging, reducing overfitting while improving stability and robustness.

TASK: Include code snippet(s) also be sure explain why you chose the values of hyperparameters for the methodology of baseline models

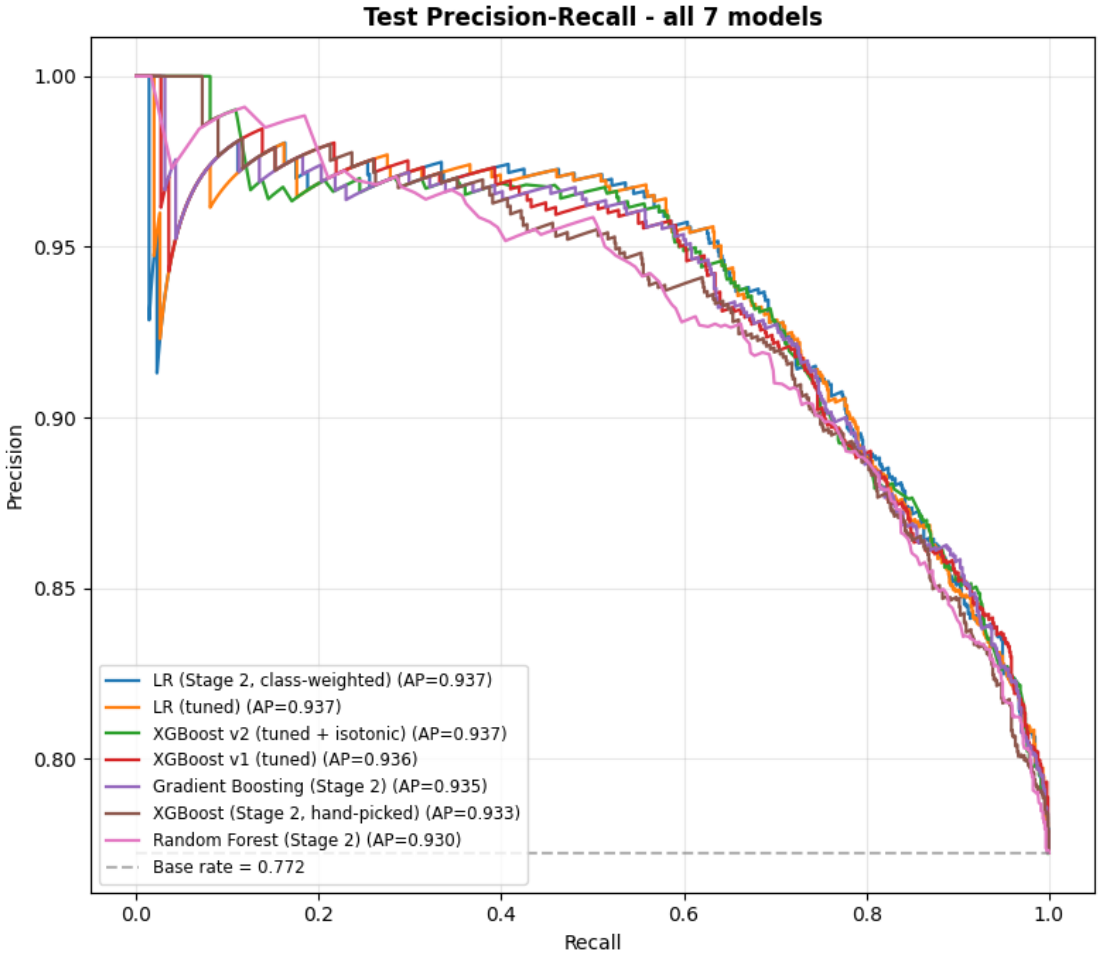
The gradient boosting baseline model was implemented as a sequential ensemble method that iteratively corrects errors from previous trees. Because boosting methods are capable of modeling subtle nonlinear patterns and interactions, gradient boosting served as a strong benchmark between traditional ensemble methods and more advanced boosting algorithms such as XGBoost.

TASK: Include code snippet(s) here

Finally, the XGBoost baseline model was implemented using hand-selected hyperparameters derived from initial experimentation and prior literature. XGBoost extends gradient boosting with regularization, optimized tree construction, and efficient parallelization, making it one of the strongest classification algorithms for structured tabular datasets.

TASK: Include code snippet(s) here

After comparing the Stage 2 baseline results, logistic regression and XGBoost were selected for optimization in Stage 3 because they demonstrated the strongest overall potential. Logistic regression achieved the highest validation ROC-AUC and PR-AUC scores among all baseline models, showing that a simpler interpretable model already captured much of the predictive signal in the dataset. XGBoost was selected because it provided the largest hyperparameter tuning surface and the greatest opportunity for improvement through regularization, calibration, threshold tuning, and advanced ensemble optimization. Together, these two models represented the strongest balance between interpretability and predictive performance.



For stage 3, we optimized the logistic regression model using GridSearchCV and capacity-based thresholding. To begin, we tuned the model's hyperparameters by defining a parameter grid that included C, logistic_l1_ratio, logistic_solver, and logistic_class_weight. The parameter C controls the strength of regularization, where smaller values impose stronger regularization. The l1_ratio determines the type of penalty applied: a value of 0 corresponds to ridge (L2), a value of 1 corresponds to lasso (L1), and intermediate values represent elastic net. The solver specifies the optimization algorithm used to minimize the logistic regression loss; we selected saga because it is the only solver that supports elastic net regularization through l1_ratio. Finally, class_weight adjusts how much the model penalizes errors for each class. Because our target label is highly imbalanced, we set class_weight to "balanced," so the model gives more weight to the minority class.

For the grid search, we used average precision (PR-AUC) as the scoring metric, since the dataset is imbalanced PR-AUC better evaluates performance on the positive class by measuring the precision-recall tradeoff across thresholds. Below is a representation of our group's workflow.

[ ]:
```
param_grid = {
    "logistic__C": [0.001, 0.01, 0.1, 1, 10, 100],
    "logistic__l1_ratio": [0, 0.25, 0.5, 0.75, 1],    # 0 = L2, 1 = L1
```

```
    "logistic__solver": ["saga"],   # saga is the only solver supporting␣
 ↪l1_ratio
    "logistic__class_weight": [None, "balanced"]
}

grid = GridSearchCV(
    estimator=pipeline,
    param_grid=param_grid,
    scoring="average_precision", # Class label is highly imbalanced so this is␣
 ↪the optimal scoring metric
    cv=5,
    n_jobs = -1,
    verbose=0
)
```

The next step was to tune the classification threshold on the validation set using a capacity-based threshold after completing the initial training on the training set. To apply this method, we selected the best-performing model from our GridSearchCV results and chose a capacity level of 60%. This means the model was allowed to flag at most 60% of patients as high-risk. We selected this value because, in a realistic clinical setting, flagging a much higher proportion would lead to over-classification and overwhelm available resources. A 60% capacity provides a practical balance: it allows the model to identify a substantial portion of potentially high-risk patients without producing an operationally unrealistic number of alerts. Below is the implementation of this approach.

[ ]:
```
# Capacity-based threshold tuning (top-K)
best_model = grid.best_estimator_
capacity = 0.60  # flag top 60%
# Get validation probabilities
y_val_probs = best_model.predict_proba(X_val)[:, 1]
# Compute threshold at the (1 - capacity) percentile
optimal_threshold = np.percentile(y_val_probs, 100 * (1 - capacity))
```

We evaluated the final model on the held-out test set using ROC-AUC, PR-AUC, Brier score loss, F2-score, accuracy, the confusion matrix, and the classification report. Because this project focuses on predicting 30-day hospital readmissions, our priority is minimizing false negatives. Missing a high-risk patient (a false negative) is clinically more harmful than over-flagging patients who may not be readmitted.

ROC-AUC measures the model's ability to rank positive cases above negative ones. A value of 0.83 indicates that the model reliably separates readmissions from non-readmissions across thresholds.

PR-AUC focuses on the precision–recall tradeoff for the positive class. With a PR-AUC of 0.94, the model is extremely effective at identifying true readmissions even under class imbalance.

The F2 -score weights recall twice as heavily as precision, reflecting our emphasis on avoiding false negatives. A score of 0.77 shows that the model performs well in recall-dominant evaluation, aligning with clinical priorities.

The Brier score evaluates the accuracy and calibration of predicted probabilities. A value of 0.17

indicates that the model's probability estimates are reasonably well-calibrated for clinical risk interpretation.

Accuracy measures the proportion of correct predictions overall. With an accuracy of 0.733, the model performs moderately well, though accuracy is less informative in imbalanced medical datasets.

A confusion matrix provides a breakdown of the model's predictions compared to the actual outcomes. Based on the confusion matrix results, the model correctly identifies 81% of negatives and 71% of positives.

The classification report provides precision, recall, and F-scores for each class. For the positive class, precision is strong, but recall is 0.71, reinforcing that nearly one-third of true readmissions are not captured.

In this evaluation, we identified the three variables with the strongest positive impact on the predicted readmission risk by examining the largest logistic regression coefficients. These coefficients represent the change in log-odds of readmission for a one-unit increase in each variable. The top contributors were: the number of previous readmissions (num___prev_readmissions), a primary diagnosis of COPD (primary_diagnosis_COPD), and a primary diagnosis of stroke(primary_diagnosis_Stroke), each substantially increasing the model's estimated risk.

The model is highly confident in its predictions of positive cases, but it still misses a non-trivial portion of true readmissions. This behavior is intentional: in a realistic hospital setting, the goal is to avoid under-preparing for potential readmissions. Prioritizing recall ensures that most high-risk patients are flagged, even if this increases false positives.

=== FINAL TEST SET PERFORMANCE ===

Top 3 coeffcients with the strongest impact:
feature coefficient odds_ratio

num___prev_readmissions 0.669271 1.952814

cat___primary_diagnosis_COPD 0.600718 1.823427

cat___primary_diagnosis_Stroke 0.468534 1.597650

ROC-AUC: 0.8299410046982862

PR-AUC: 0.9370646421717862

Brier Score: 0.1706477106868414

F2-score: 0.7657995888084383 Accuracy: 0.7333333333333333 Confusion Matrix: [[221 52] [268 659]]

Classification Report: precision recall f1-score support

0	0.45	0.81	0.58	273
1	0.93	0.71	0.80	927
accuracy			0.73	1200

macro avg 0.69 0.76 0.69 1200 weighted avg 0.82 0.73 0.75 1200

Percentage of patients flagged as high-risk: 59.25%

TASK: XGBoost optimization details/ methodology

Second optimized model implementation details: Write here

The final model selected was the tuned XGBoost classifier with isotonic calibration. Although the optimized logistic regression achieved the highest ROC-AUC, the margin was extremely small and not practically meaningful. In contrast, XGBoost v2 outperformed the other models on the metrics most relevant to our clinical objective. It achieved the highest PR-AUC, the lowest Brier score by a substantial margin, and the highest F2 score. Most importantly, it produced the fewest false negatives and the most true positives, meaning it identified more actual readmissions while missing fewer high-risk patients. Because minimizing false negatives is critical in this clinical setting, XGBoost v2 provides the best balance of discrimination, calibration, and recall-oriented performance.

[3]:
```
import pandas as pd

url = "https://raw.githubusercontent.com/alex-115/CSCI-485-project/main/
 ↪Himanshu_Stage3/final_comparison/metrics.csv"
df = pd.read_csv(url)
df
```

[3]:

	model	test_roc_auc	test_pr_auc	test_brier	test_f2	threshold	tn	fp	fn	tp
0	Logistic Regression (tuned)	0.829941	0.937065	0.170648	0.934774	0.197192	67	206	27	900
1	XGBoost v2 (tuned + isotonic)	0.826612	0.937329	0.130424	0.936271	0.356988	53	220	22	905
2	XGBoost v1 (tuned)	0.825191	0.935967	0.169722	0.929916	0.211445	90	183	38	889

Now that we have selected the final model, it is essential to understand its overall behavior. To do this, we begin with SHAP analysis. SHAP is a model interpretation method that quantifies how each feature contributes to individual predictions as well as to the model's overall behavior. Using the shap library, we first extracted the final tuned XGBoost model and constructed a SHAP explainer with the TreeExplainer, which is appropriate for tree-based models such as XGBoost. We then computed SHAP values and generated a beeswarm plot to visualize the distribution and density of feature impacts across all observations. This plot highlights how strongly each feature influences the model and in which direction. We also produced a mean absolute SHAP value bar plot to provide a clear ranking of the most influential features in the model's predictions. Below is the implementation.

[ ]:
```
# Extract the trained XGB model
xgb_model = tuned_model.named_steps["xgb"]

# Extract the preprocessor and transform the test set
preprocessor = tuned_model.named_steps["preprocess"]
X_test_t = preprocessor.transform(X_test)
```

```
# Build SHAP explainer
explainer = shap.TreeExplainer(xgb_model)

# Compute SHAP values for the test set
shap_values = explainer.shap_values(X_test_t)
print("shap values: \n")
feature_names = tuned_model.named_steps["preprocess"].get_feature_names_out()
# feature importance
mean_abs_shap = np.abs(shap_values).mean(axis=0)
```

The shap analysis revealed that num__prev_admissions, num_age, num_comorbidities_count, num_length_stay, and num_medication_count were the top 5 highest shap values, meaning they had the higehst contribition to the models overall behavior.

The SHAP beeswarm plot not only confirms the top five contributors to the model's predictions but also reveals how each feature influences individual patient-level outcomes. The feature num___prev_readmissions shows a distinctly right-skewed distribution of SHAP values: most observations have SHAP values above 0.5, indicating that prior readmissions consistently push the model toward predicting a higher risk of 30-day readmission.

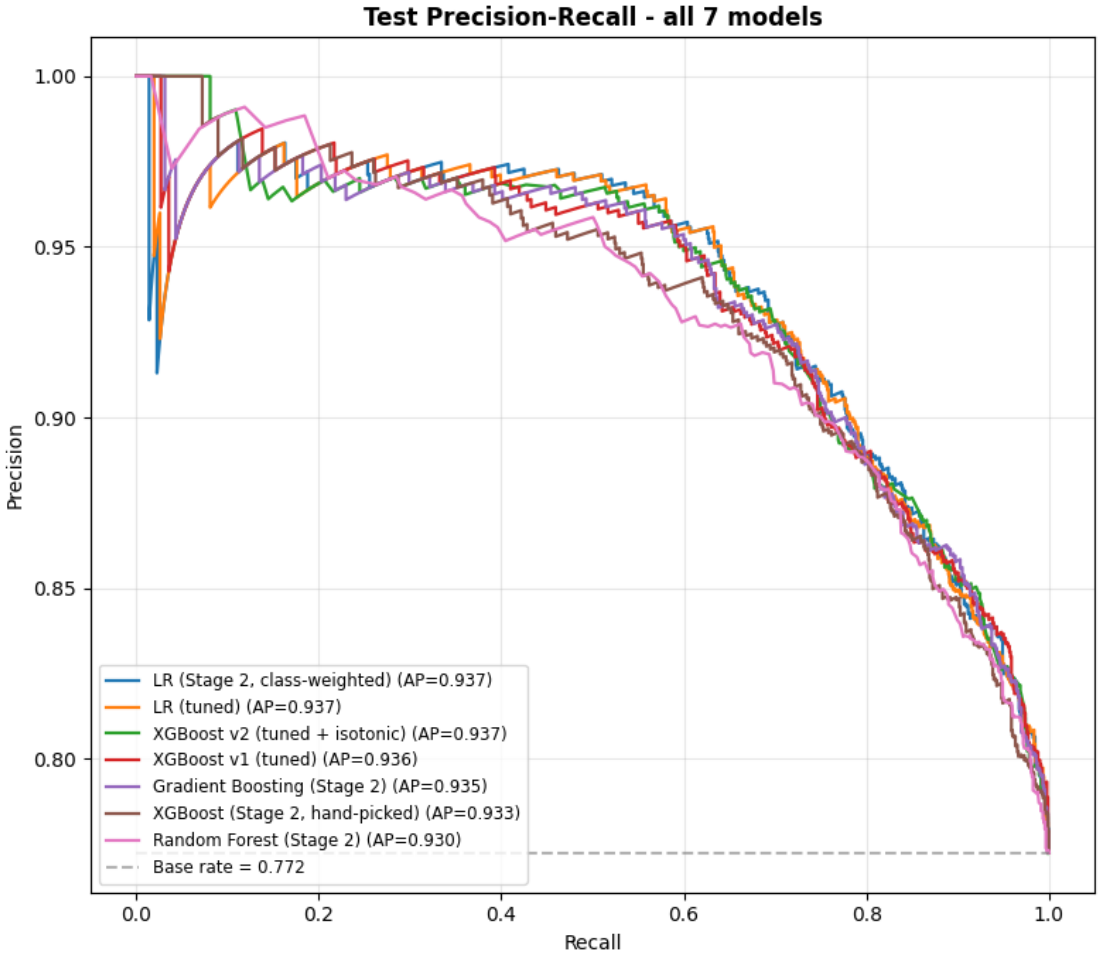
In contrast, num___age, the second-strongest predictor, displays a more balanced spread of SHAP values across the positive and negative range. While older ages generally increase predicted risk, the plot also shows a noticeable concentration of negative SHAP values around –0.5. This suggests that for some patients, age interacts with other clinical factors in a way that reduces predicted readmission risk. However, the overall distribution still leans toward positive SHAP values, meaning age typically contributes to higher predicted risk even though its effect is more mixed than that of previous readmissions.

Results and analysis

The final results showed that all models achieved relatively similar ranking performance, with ROC-AUC values clustered within a narrow range. Among the baseline models, class-weighted logistic regression achieved the strongest overall ROC-AUC performance, while XGBoost and gradient boosting produced comparable PR-AUC results. This suggests that the dataset's predictive signal could already be captured effectively using simpler linear methods.

After tuning, both logistic regression and XGBoost improved in recall-focused performance and probability estimation. The optimized logistic regression model achieved a ROC-AUC of approximately 0.83 and a PR-AUC of approximately 0.94 on the held-out test set. The model also achieved strong recall for the positive class, correctly identifying 659 out of 927 true readmissions using the selected capacity-based threshold. This behavior aligns with the project's primary objective of minimizing false negatives.

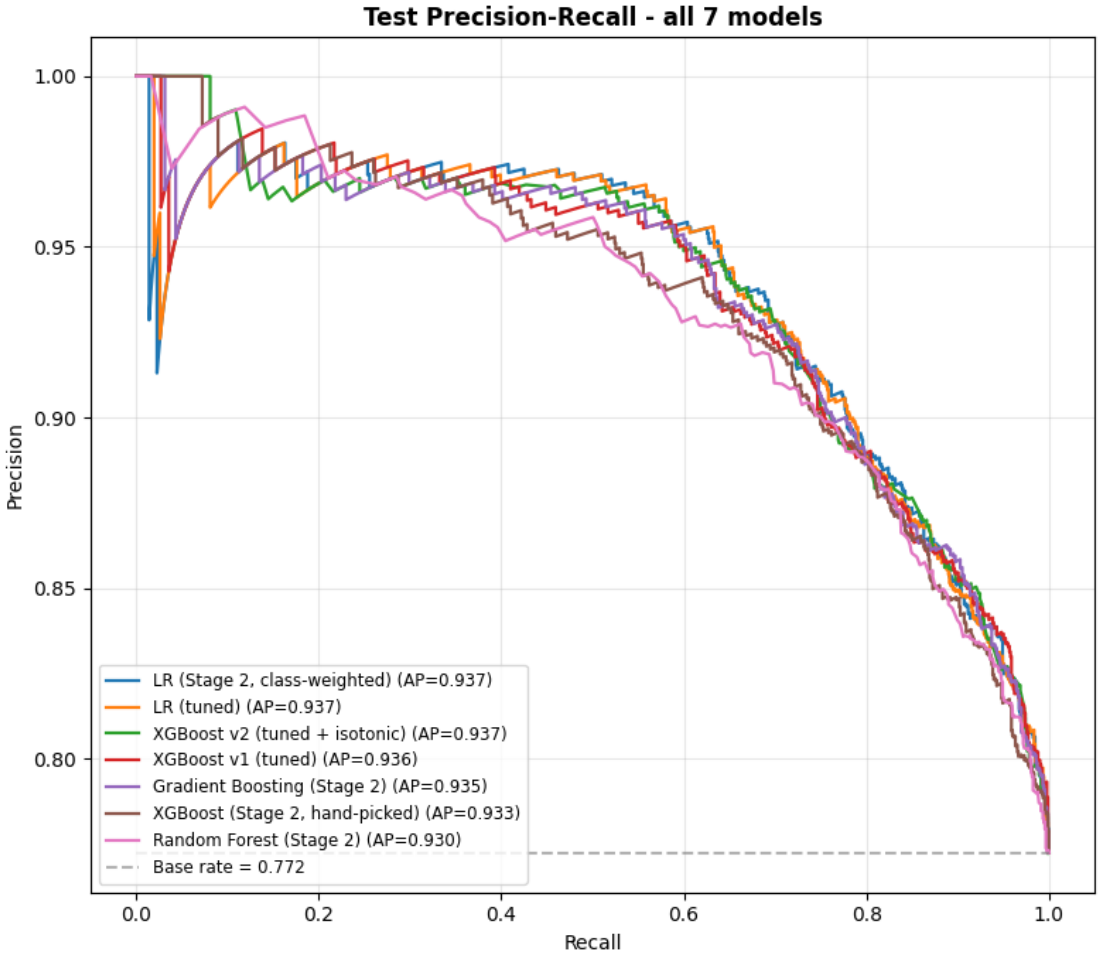
The tuned XGBoost model produced comparable ROC-AUC and PR-AUC performance while providing stronger probability calibration after isotonic calibration was applied. This improvement was reflected by the lower Brier score achieved by the calibrated XGBoost model. As a result, XGBoost generated more reliable predicted probabilities, making it more appropriate for hospital scenarios where patients may need to be grouped into different risk categories.



One major finding was that increasing model complexity did not dramatically improve ranking performance. Random forest, gradient boosting, and XGBoost all performed similarly to logistic regression on ROC-AUC and PR-AUC metrics. This indicates that the dataset likely contains relatively strong linear relationships between predictors and readmission risk, limiting the gains achievable through more complex nonlinear models.

What worked especially well was the use of stratified splitting, preprocessing pipelines, class balancing, and threshold tuning. Threshold optimization significantly improved recall for high-risk patients compared to using the default 0.5 threshold. In contrast, what did not improve substantially was the separation performance between model families, as all models converged to similar ROC-AUC values.

Overall, the project demonstrates that both interpretable linear models and advanced boosting methods can effectively predict hospital readmissions. Logistic regression provided simplicity and interpretability, while XGBoost offered superior probability calibration and more flexible nonlinear modeling.



Conclusion

The goal of this project was to develop machine learning models capable of predicting whether a patient would experience an unplanned hospital readmission within 30 days using structured hospital data.

Overall, the results demonstrated that both logistic regression and XGBoost were highly effective at predicting readmission risk. Logistic regression achieved the strongest overall ROC-AUC performance while remaining highly interpretable, whereas XGBoost produced the best probability calibration after isotonic calibration was applied. Across all models, previous readmissions, COPD diagnosis, stroke diagnosis, age, and comorbidities consistently emerged as the strongest predictors of readmission risk.

These results are important because hospital readmissions are closely tied to healthcare quality, resource allocation, patient outcomes, and operational planning. A reliable readmission prediction model can help hospitals identify high-risk patients earlier, prioritize follow-up care, improve discharge planning, and better prepare staffing and bed capacity. In practice, prioritizing recall helps ensure that fewer high-risk patients are missed, even if additional low-risk patients are incorrectly flagged.

One important limitation of this project is that the dataset used was synthetic rather than collected

from real hospital systems. Synthetic data is artificially generated and may not fully represent the complexity, variability, and unpredictability of real clinical environments. Real patient outcomes are influenced by many additional factors such as laboratory results, medication history, socioeconomic conditions, physician decision-making, and longitudinal health records, none of which were available in this dataset. As a result, although the models performed strongly on this dataset, their real-world clinical performance may differ and would require external validation using real patient data before deployment in healthcare settings.

Future work

For future work, this model could be improved by incorporating cost-sensitive learning or custom loss functions that penalize false negatives more heavily than false positives. In our project, missing a high-risk patient incurs a substantially higher cost than incorrectly flagging a low-risk patient. Cost-sensitive learning embeds this idea directly into the training objective by assigning a larger weight to errors made on the positive class. As a result, the model is discouraged from predicting negative when the true label is positive.

By making false negatives "more expensive," the model shifts the decision boundary toward predicting the positive class more often. This encourages higher recall, improves the model's ability to detect true readmissions, and reduces the likelihood of missing high-risk patients. Although this approach may increase the number of false positives, this tradeoff is often acceptable in healthcare contexts where early intervention is preferable. If there were no limitations on data due to medical privacy, the model would perform significantly better with a dataset containing more clinically meaningful features, such as medication history, time-series vital signs, or lab results.

For a future model, it would be optimal to evaluate with more complex models to improve recall, such as stack ensembles combining a linear model with tree-based models. A stacked ensemble can improve recall because each model captures different types of structure in the data, and the meta-learner learns how to weight them in a way that reduces false negatives. Linear models excel at identifying broad trends while tree-based models capture nonlinear interactions and complex threshold effects that a linear model cannot represent. When these prediction patterns are combined, the ensemble produces a more sensitive classifier that is better at identifying subtle indicators of positive cases. If tree-based models tend to detect certain high-risk subgroups and the linear model performs better on others, the meta-learner can assign higher weight to whichever model is more reliable for predicting the positive class. This reduces the likelihood of false negatives and shifts the final decision boundary toward higher recall. As a result, the stacked ensemble becomes more effective at identifying true readmissions than an individual model alone.

If this project were to continue development, the goal would be integrations within hospital operations. A next step would be to deploy the model as part of a real-time risk dashboard that continuously evaluates incoming patients and updates their predicted likelihood of 30-day readmission. This dashboard could be used by clinical staff, case managers, and hospital administrators to identify high-risk patients early and allocate resources more effectively.

Beyond patient-level predictions, the model could also support monthly operational planning. By aggregating predicted readmission risks across all patients admitted during a given month, the system could estimate the expected number of readmissions for the following month. Hospitals could use these forecasts to anticipate demand for supplies, beds, staffing, and follow-up care resources. This planning would help prevent shortages that could impact patient care and allow hospitals to be prepared for a high unplanned readmission of patients.

References

scikit-learn

Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., … Duchesnay, É. (2011).
Scikit-learn: Machine Learning in Python. Journal of Machine Learning Research, 12, 2825–2830.

Matplotlib

Hunter, J. D. (2007). Matplotlib: A 2D graphics environment. Computing in Science & Engineering, 9(3), 90–95.

pandas

McKinney, W. (2010). Data structures for statistical computing in Python. In S. van der Walt & J. Millman (Eds.), Proceedings of the 9th Python in Science Conference (pp. 51–56).

Numpy

Harris, C. R., Millman, K. J., van der Walt, S. J., Gommers, R., Virtanen, P., Cournapeau, D., … Oliphant, T. E. (2020). Array programming with NumPy. Nature, 585, 357–362.

Shap

Lundberg, S. M., & Lee, S.-I. (2017). A unified approach to interpreting model predictions. In Advances in Neural Information Processing Systems (pp. 4765–4774)